

ARES Protocol — Full Reference

Every endpoint, every transport: REST/JSON, REST/XML, peerbus

REST/JSON

XML

peerbus

This is the complete reference. If you only drive a robot, the short **PROTOCOL.typ** quick-start (register / heartbeat / claim / release) is all you need. This document adds the read/query endpoints, route planning, scheduling, lease administration, and the per-transport details.

Two tiers. **Tier-1** = the four flat, key-authenticated robot calls — flat scalar bodies, a **decision** + **reason** reply. **Tier-2** = richer endpoints for graph-aware clients, with fuller JSON/XML payloads. Read-only endpoints are open; every state-changing endpoint takes a key.

Contents

1	Connection points	2
2	Shared rules	2
2.1	Resource identifiers	2
2.2	Authentication key	2
2.3	Proving a did:key	2
2.4	Replies and errors	3
3	Tier-1 flat protocol	3
3.1	Register — POST /ares/v1/robots	3
3.2	Heartbeat — POST /ares/v1/robots/{id}/heartbeat	3
3.3	Claim — POST /ares/v1/claims/{zone node edge}	3
3.4	Release — POST /ares/v1/leases/release/{zone node edge}	3
4	Tier-2 endpoints	3
4.1	Map and fleet (all read-only)	4
4.2	Route planning — POST /routes/plan (read-only)	4
4.3	Robots	4
4.4	Claims	4
4.5	Leases	5
5	Transports	5
5.1	Per-operation address map	5
5.2	REST/JSON and XML	6
6	Reason codes	6
7	Wire types	6
8	Operator options (opt-in)	7
8.1	State persistence	7
8.2	Admin endpoint authentication	7

1 Connection points

Transport	How to reach it
REST/JSON	<code>http://<host>:8080/ares/v1/...</code> , <code>Content-Type: application/json</code> .
REST/XML	Same URLs; send/accept <code>application/xml</code> . The REST server answers XML when the request sets the XML content-type/accept header, and there is also a standalone XML router.
peerbus	The canonical req/res bus every adapter speaks. Typed datapod messages on the topics <code>ares/v1/...</code> ; see <code>docs/WRITING_ADAPTER.md</code> .

2 Shared rules

2.1 Resource identifiers

Zones, nodes, and edges are UUIDs internally; client calls may use the UUID or a numeric alias. A robot id may be an integer or a UUID string. In JSON, send numeric ids/keys as quoted strings so JSON and XML agree (XML has no number type).

```
"100" // numeric alias
"00000000-0000-0000-0000-00000000100" // UUID
```

2.2 Authentication key

A `key` authenticates the acting robot. Forms:

Key form	Meaning
integer, e.g. <code>1234</code>	Simple numeric password.
<code>pass:<secret></code>	Password; everything after the prefix.
<code>did:key:<multibase></code>	Ed25519 identity. Binds at registration but authorises nothing until proved (§2.3).
<code>tok:<token></code>	Bearer token issued by <code>/auth/prove</code> ; valid one hour.

- The key is set at registration and bound to the robot id; later calls resend it and the server checks it against that robot.
- Optional on the flat (tier-1) calls:** omit it and the server uses a shared default password — convenient but insecure. A robot that registered with a real key must keep sending it.
- Required on state-changing tier-2 calls** (assign-route, schedule, the nested claim submit) — in the request body. **Read-only** calls (health, snapshot, map lookups, list/get, route planning, claim **evaluate**) take no key.
- Admin mutation endpoints** (unregister, remove claim, release/add lease) are **open by default**; a key is required only when the operator opts in with `SYNCBOT_ADMIN_AUTH` (§8.2).

2.3 Proving a `did:key`

A public key is not a secret, so sending one proves nothing. A `did:key` robot signs a server nonce instead and trades the signature for a bearer token.

Call	Body / reply
<code>POST /auth/challenge</code>	<code>{"robot":"7"} → {"nonce":"<hex>","expires_in":30}</code> . Open, and accepts an unregistered id: an identity must be provable before it is bound, so nobody can squat one.
<code>POST /auth/prove</code>	<code>{"robot":"7","did":"did:key:...", "signature":"<hex>"}</code> → <code>{"token":"...", "expires_in":3600}</code> . The signature is over the raw nonce bytes.

Later calls send the token in the ordinary `key` field as `tok:<token>`. Registering a bare `did:key` with no proof is refused (reason 5).

2.4 Replies and errors

Tier	Reply shape
Tier-1 (flat)	<code>decision</code> (<code>1</code> ok/grant, <code>0</code> deny) + <code>reason</code> (enum); claims add <code>blocked</code> (offending id) on denial. In XML the reply root is <code><reply></code> ; the request's outer tag is ignored on input.
Tier-2	The endpoint's data type as JSON/XML. On error: HTTP <code>400</code> with <code>{"message": "..."} .</code>

Reserved reason codes — same on every flat call: `0` = OK, `1` = mismatched key. Endpoint-specific reasons start at `2` (see §6).

3 Tier-1 flat protocol

The four robot calls. Type is in the address; body is flat scalars; reply is `decision` + `reason`. Available on every transport — XML shown; JSON and peerbus carry the same fields (see §5).

3.1 Register — `POST /ares/v1/robots`

Optional `<alive>` = heartbeat interval in seconds (default 2); if no heartbeat arrives for `2×` that, the server marks the robot inactive and **auto-releases all its claims** (a background sweeper frees zones held by crashed/disconnected robots, so nothing stays stuck).

```
<reg><robot>7</robot><key>1234</key><alive>2</alive></reg>  <!-- key/alive optional; robot int or UUID -->
<reply><decision>1</decision><reason>0</reason></reply>
```

3.2 Heartbeat — `POST /ares/v1/robots/{id}/heartbeat`

Liveness + position (`zone` / `node` / `edge`); the server stamps the tick. A non-negative `zone` is a zone id; `zone = -1` means “unknown / not in any claimed zone”.

```
<hb><key>1234</key><zone>42</zone></hb>  <!-- or <zone>-1</zone> if unknown -->
<reply><decision>1</decision><reason>0</reason></reply>
```

3.3 Claim — `POST /ares/v1/claims/{zone|node|edge}`

Repeat `<id>` to claim several atomically (all-or-nothing). Optional `<access_mode>` (0 = unspecified → exclusive; 1 = exclusive default; 2 = shared; 3+ reserved → rejected) and `<lease_time>` (seconds; 0 = unlimited).

```
<claim><key>1234</key><robot>7</robot><id>42</id><id>43</id>
  <access_mode>1</access_mode><lease_time>30</lease_time></claim>
<reply><decision>0</decision><reason>2</reason><blocked>43</blocked></reply>
```

Claiming a zone reserves everything inside it, so a zone claim conflicts with a node/edge claim within that zone (and vice versa). A zone with capacity greater than 1 may be held concurrently by several robots under `access_mode=2` (shared), up to that capacity; a further shared claim past capacity is denied with reason `3` (**capacity exceeded**), while an **exclusive** claim (`access_mode=1`) on a shared zone still conflicts (reason `2`).

3.4 Release — `POST /ares/v1/leases/release/{zone|node|edge}`

```
<rel><key>1234</key><robot>7</robot><id>42</id></rel>
<reply><decision>1</decision><reason>0</reason></reply>
```

4 Tier-2 endpoints

Richer endpoints for graph-aware clients. REST paths shown; the peerbus topic for each is in §5.1. `R` = read-only (open), `W` = state-changing (needs `key`).

4.1 Map and fleet (all read-only)

	Endpoint	Returns
R	GET /health	{status, version}
R	GET /fleet/snapshot	{robots[], requests[], leases[]}
R	GET /zones . GET /zones/{id}	ZoneView (id, numeric_id, name, kind, parent, children, nodes, properties)
R	GET /nodes . GET /nodes/{id}	NodeView (id, numeric_id, name, position, zones, properties)
R	GET /edges . GET /edges/{id}	EdgeView (id, numeric_id, source, target, directed, weight, zones, properties)

4.2 Route planning — POST /routes/plan (read-only)

```
request  {"start_node_id":"1001","goal_node_id":"1003","use_penalties":false}
response {"found":true,"distance":12.0,"plan":{...},"failure":null}
```

`plan` is a `RoutePlan` (traversed node/edge/zone ids + per-step costs); `failure` carries diagnostics when no route is found.

4.3 Robots

	Endpoint	Notes
W	POST /robots	Flat register (§3). Body <code>robot</code> + optional <code>key</code> .
R	GET /robots . GET /robots/{id}	<code>RobotState</code> / list.
W	DELETE /robots/{id}	Unregister. Admin path; open by default, owning-robot key via <code>?key=</code> when <code>SYNCBOT_ADMIN_AUTH</code> is set (§8.2).
W	POST /robots/{id}/heartbeat	Flat heartbeat (§3).
W	POST /robots/{id}/route	Assign a precomputed <code>RoutePlan</code> . Body <code>AssignRouteRequest</code> + <code>key</code> .
W	POST /robots/{id}/schedule	Schedule from a claim → <code>ScheduleDecision</code> (Proceed / Queue / Replan). Body <code>ScheduleRobotRouteRequest</code> + <code>key</code> .

```
// assign route
{"route_plan":{...},"horizon":100,"updated_at_tick":10,"key":"1234"}
// schedule
{"claim_id":10,"start_tick":20,"ticks_per_cost_unit":1.0,"access_mode":"Exclusive","key":"1234"}
```

4.4 Claims

	Endpoint	Notes
R	GET /claims . GET /claims/{id}	Active claim requests.
R	POST /claims/evaluate	Dry-run a nested claim; does not store it. No key.
W	POST /claims	Nested submit: evaluate + store if granted. Body <code>ClaimRequestWire</code> + <code>key</code> .
W	POST /claims/{zone,node,edge}	Flat claim (§3).
W	DELETE /claims/{id}	Remove a request. Admin path; open by default, owning-robot key via <code>?key=</code> when <code>SYNCBOT_ADMIN_AUTH</code> is set (§8.2).

Nested `ClaimRequestWire` (for `evaluate` / `POST /claims`):

```
{
  "id": 10,
  "robot_id": 1,
  "key": "1234",
  "access_mode": "Exclusive",
  "priority": 10,
  "requested_at_tick": 10,
  "window": {
    "start_tick": 10,
    "end_tick": 100
  },
  "targets": [
    {
      "kind": "Zone",
      "resource_id": "100"
    }
  ]
}
```

`evaluate` returns the full `ClaimEvaluation` (`decision`, `reason`, `conflicting_*`, `blocking_target`, `diagnostics`);
`POST /claims` returns the same and stores the request when `decision == Grant`.

4.5 Leases

	Endpoint	Notes
R	<code>GET /leases</code>	Active leases.
W	<code>POST /leases</code>	Add a lease directly (admin / integration).
W	<code>POST /leases/release</code>	Release by lease id — body <code>{"lease_id":501,"released_at_tick":50}</code> .
W	<code>POST /leases/release/{zone,node,edge}</code>	Flat release by robot + resource (§3).
W	<code>DELETE /leases/{id}</code>	Release by lease id in the path.

5 Transports

5.1 Per-operation address map

The same operation has one address per transport. REST uses HTTP verb + path; peerbus uses a topic. Where REST puts an id in the URL, peerbus puts it in the message body.

Operation	REST path	peerbus topic
health	<code>GET /health</code>	<code>ares/v1/health</code>
snapshot	<code>GET /fleet/snapshot</code>	<code>ares/v1/fleet/snapshot</code>
list/get zone	<code>GET /zones/{id}</code>	<code>ares/v1/zones/list</code> . <code>.../get</code>
list/get node	<code>GET /nodes/{id}</code>	<code>ares/v1/nodes/list</code> . <code>.../get</code>
list/get edge	<code>GET /edges/{id}</code>	<code>ares/v1/edges/list</code> . <code>.../get</code>
plan route	<code>POST /routes/plan</code>	<code>ares/v1/routes/plan</code>
register	<code>POST /robots</code>	<code>ares/v1/robots/register</code>
list/get robot	<code>GET /robots/{id}</code>	<code>ares/v1/robots/list</code> . <code>.../get</code>
unregister	<code>DELETE /robots/{id}</code>	<code>ares/v1/robots/unregister</code>
heartbeat	<code>POST /robots/{id}/heartbeat</code>	<code>ares/v1/robots/heartbeat</code>
assign route	<code>POST /robots/{id}/route</code>	<code>ares/v1/robots/assign_route</code>
schedule	<code>POST /robots/{id}/schedule</code>	<code>ares/v1/robots/schedule</code>
list/get claim	<code>GET /claims/{id}</code>	<code>ares/v1/claims/list</code> . <code>.../get</code>
evaluate claim	<code>POST /claims/evaluate</code>	<code>ares/v1/claims/evaluate</code>

Operation	REST path	peerbus topic
submit claim	POST /claims	ares/v1/claims/request
flat claim	POST /claims/{kind}	ares/v1/claims/{kind}
remove claim	DELETE /claims/{id}	ares/v1/claims/remove
list lease	GET /leases	ares/v1/leases/list
add lease	POST /leases	ares/v1/leases/add
release lease	POST /leases/release	ares/v1/leases/release
flat release	POST /leases/release/{kind}	ares/v1/leases/release/{kind}

On peerbus, calls that REST addresses by URL id carry the id as a body field instead — `id`, `robot_id`, or `claim_id`. The robot id on a flat peerbus call is likewise a body field, since there is no URL.

5.2 REST/JSON and XML

All REST bodies and replies are JSON by default. The same routes speak XML when the request sets `Content-Type: application/xml` (and `Accept: application/xml` for GETs). XML is encoded by `quick-xml`; on input the outer element name is ignored. JSON note: the flat claim `id` is an array (`"id": [42]`, or `[42, 43]`); XML repeats `<id>`.

6 Reason codes

`0` = OK and `1` = mismatched key on every flat call. Endpoint-specific:

Call	Reasons (≥ 2)
register	2 already registered · 3 bad id (not int/UUID) · 4 unsupported key scheme
heartbeat	2 not registered
claim	2 conflict · 3 capacity exceeded · 4 unknown resource · 5 bad request (no id / unsupported <code>access_mode</code>)
release	2 no such lease · 3 unknown / bad

Tier-2 endpoints report failure as HTTP `400` + `{"message": "..."}` rather than a reason code.

7 Wire types

Type	Fields
FlatRegister	robot, key?, alive? (heartbeat interval s, default 2)
FlatHeartbeat	key?, one of zone / node / edge
FlatClaim	key?, robot, id[], access_mode?, lease_time?
FlatRelease	key?, robot, id
FlatReply	decision (1/0), reason (enum), blocked?
PlanRouteRequest	start_node_id, goal_node_id, use_penalties
ClaimRequestWire	id, robot_id, access_mode, priority, window, targets[], key?
AssignRouteRequest	route_plan, horizon, updated_at_tick, key?
ScheduleRobotRouteRequest	claim_id, start_tick, ticks_per_cost_unit, access_mode, key?
ReleaseLeaseRequest	lease_id, released_at_tick?

Common enums: claim target kind `Zone / Node / Edge`; access mode `Exclusive / Shared`; lease disposition `Active / Released / Expired / Revoked`; robot progress `Idle / FollowingRoute / Waiting / Queued / Blocked / Replanning`.

8 Operator options (opt-in)

Two server-side options that an operator may enable at deploy time. **Both are off by default; the protocol seen by existing clients is unchanged.**

8.1 State persistence

By default the server is **fully ephemeral**: all state lives in memory and is lost on restart. This is the default and is unchanged.

Optionally, set the env var `SYNCBOT_STATE` to a JSON file path to persist state across restarts. When set, on boot the server **restores** registrations, auth keys, UUID→id mappings, and active claims/leases from that file; while running it **flushes every 2 seconds** (atomic write) and once more on shutdown. If `SYNCBOT_STATE` is unset, nothing is written and the server stays fully ephemeral (state lost on restart — the default).

8.2 Admin endpoint authentication

The admin mutation endpoints — unregister a robot (`DELETE /robots/{id}`), remove a claim (`DELETE /claims/{id}`), release a lease by id (`DELETE /leases/{id}`) / `POST /leases/release`), and add a lease (`POST /leases`) — are **open by default**: no key is required. This is the default and is unchanged.

Optionally, an operator may set the env var `SYNCBOT_ADMIN_AUTH=1` (also accepts `true` / `yes`) to require the owning robot's key on these endpoints — supplied as `?key=...` on the body-less DELETES and `POST /leases`, or in the body where the endpoint already takes one. With auth enabled:

- a robot that registered **without** a key stays openly manageable (it is bound to the shared default key);
- a robot that registered **with** a key is protected — a missing or wrong key is rejected (reason `1` / mismatched-key error).

Both options above are **opt-in**. Leave `SYNCBOT_STATE` and `SYNCBOT_ADMIN_AUTH` unset and the server behaves exactly as before: ephemeral state and open admin endpoints.